



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие №2

Студенты группы *ИКБО-20-23 Кузнецов Лев Андреевич*

(подпись)

Ассистент *Брайловский А.В.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ	3
ХОД РАБОТЫ	4
ОТВЕТЫ НА ВОПРОСЫ	12
ВЫВОД.....	13

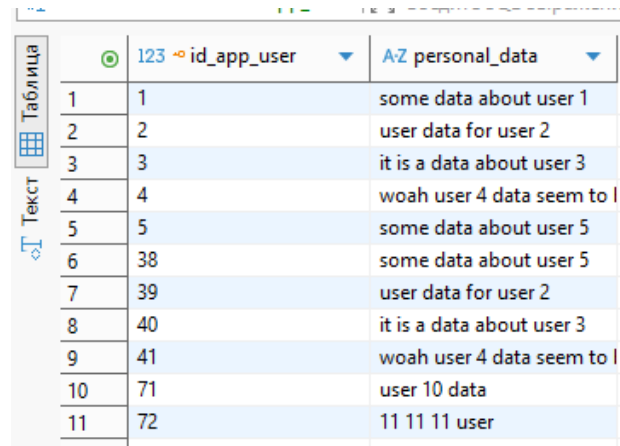
ПОСТАНОВКА ЗАДАЧИ

Цель: научиться извлекать и комбинировать данные из нескольких связанных таблиц с помощью соединений (JOIN) и теоретико-множественных операторов (UNION, INTERSECTION, EXCEPT), а также освоить продвинутые паттерны, такие как «само-соединение» и «анти-соединение».

Постановка задачи: демонстрация различных типов соединений и применение теоретико-множественных операторов.

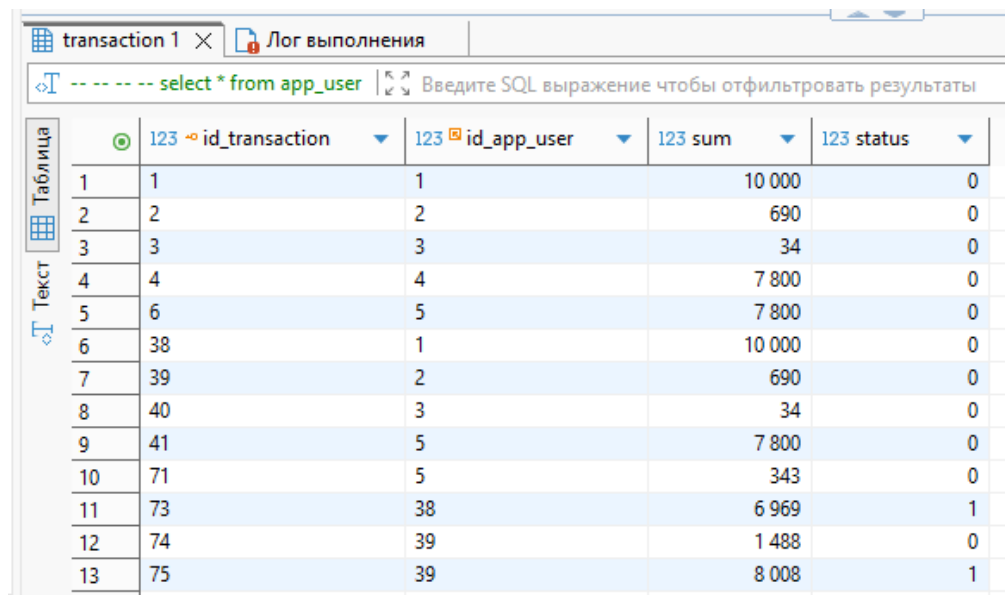
ХОД РАБОТЫ

Используемые таблицы



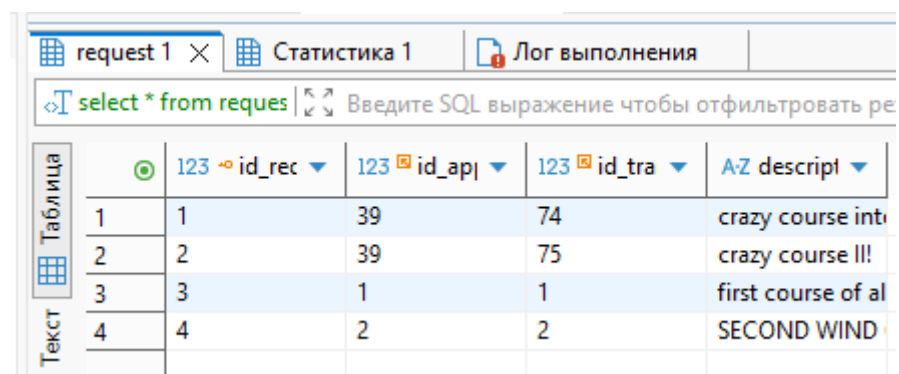
	123 id_app_user	AZ personal_data
1	1	some data about user 1
2	2	user data for user 2
3	3	it is a data about user 3
4	4	woah user 4 data seem to l
5	5	some data about user 5
6	38	some data about user 5
7	39	user data for user 2
8	40	it is a data about user 3
9	41	woah user 4 data seem to l
10	71	user 10 data
11	72	11 11 11 user

Рисунок 1 – Таблицы app_user



	123 id_transaction	123 id_app_user	123 sum	123 status
1	1	1	10 000	0
2	2	2	690	0
3	3	3	34	0
4	4	4	7 800	0
5	6	5	7 800	0
6	38	1	10 000	0
7	39	2	690	0
8	40	3	34	0
9	41	5	7 800	0
10	71	5	343	0
11	73	38	6 969	1
12	74	39	1 488	0
13	75	39	8 008	1

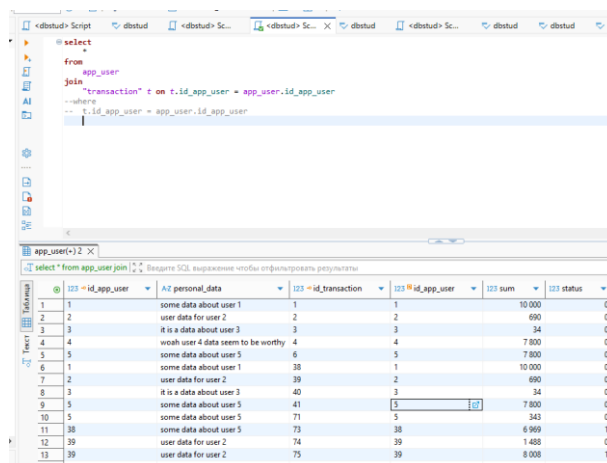
Рисунок 2 – Таблицы app_transaction



	123 id_rec	123 id_api	123 id_tra	AZ description
1	1	39	74	crazy course intro
2	2	39	75	crazy course III!
3	3	1	1	first course of al
4	4	2	2	SECOND WIND

Рисунок 3 – Таблицы request

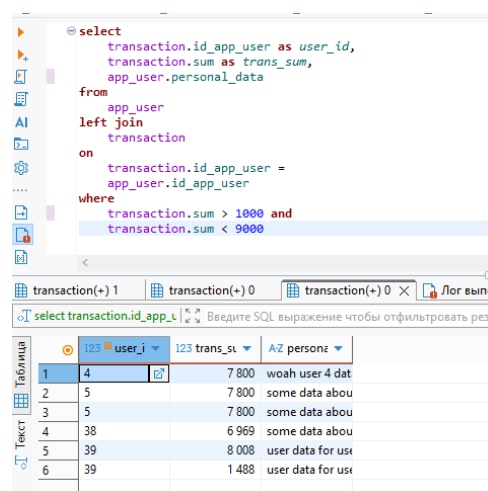
1. Основы соединения таблиц (JOIN)



```
select
from
app_user
join
transaction t on t.id_app_user = app_user.id_app_user
--where
--- t.id_app_user = app_user.id_app_user
```

	id_app_user	personal_data	id_transaction	id_app_user	sum	status
1	1	some data about user 1	1	1	10 000	0
2	2	user data for user 2	2	2	690	0
3	3	it is a data about user 3	3	3	34	0
4	4	woah user 4 data seem to be worthy	4	4	7 800	0
5	5	some data about user 5	5	5	7 800	0
6	1	some data about user 1	38	1	10 000	0
7	2	user data for user 2	39	2	690	0
8	3	it is a data about user 3	40	3	34	0
9	5	some data about user 5	41	5	7 800	0
10	5	some data about user 5	71	5	343	0
11	38	some data about user 5	73	38	6 969	1
12	39	user data for user 2	74	39	1 488	0
13	39	user data for user 2	75	39	8 008	1

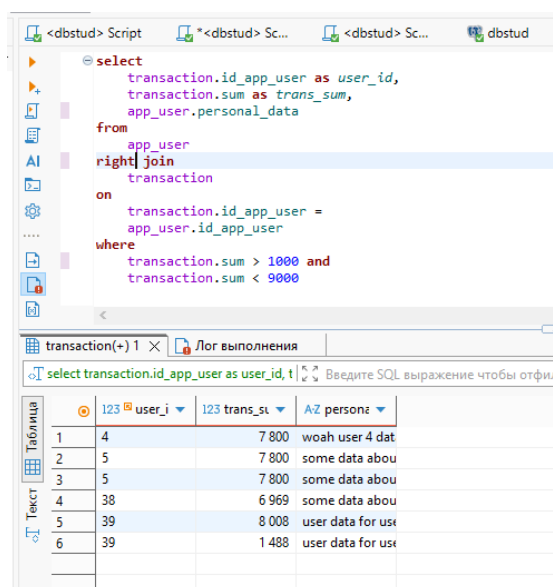
Рисунок 1.1 – Применение (inner) join



```
select
transaction.id_app_user as user_id,
transaction.sum as trans_sum,
app_user.personal_data
from
app_user
left join
transaction
on
transaction.id_app_user =
app_user.id_app_user
where
transaction.sum > 1000 and
transaction.sum < 9000
```

	user_id	trans_sum	personz
1	4	7 800	woah user 4 dat
2	5	7 800	some data abou
3	5	7 800	some data abou
4	38	6 969	some data abou
5	39	8 008	user data for use
6	39	1 488	user data for use

Рисунок 1.2 – Применение left (outer) join



```
select
transaction.id_app_user as user_id,
transaction.sum as trans_sum,
app_user.personal_data
from
app_user
right join
transaction
on
transaction.id_app_user =
app_user.id_app_user
where
transaction.sum > 1000 and
transaction.sum < 9000
```

	user_id	trans_sum	personz
1	4	7 800	woah user 4 dat
2	5	7 800	some data abou
3	5	7 800	some data abou
4	38	6 969	some data abou
5	39	8 008	user data for use
6	39	1 488	user data for use

Рисунок 1.3 – Применение right (outer) join

```

select
  transaction.sum as trans_sum,
  "transaction".id_transaction,
  request.id_app_user as request_id_app_user
from
  request
right join
  transaction
on
  transaction.id_app_user = request.id_app_user
where
  transaction.sum > 1000 and
  transaction.sum < 9000

```

	123 trans_sum	123 id_transaction	123 request_id_app_user
1	8 008	75	39
2	1 488	74	39
3	8 008	75	39
4	1 488	74	39
5	7 800	41	[NULL]
6	7 800	6	[NULL]
7	6 969	73	[NULL]
8	7 800	4	[NULL]

Рисунок 1.4 – Повторное применение right (outer) join

```

select
  transaction.sum as trans_sum,
  "transaction".id_transaction,
  request.id_app_user as request_id_app_user
from
  request
left join
  transaction
on
  transaction.id_app_user = request.id_app_user
where
  transaction.sum > 1000 and
  transaction.sum < 9000

```

	123 trans_sum	123 id_transaction	123 request_id_app_user
1	8 008	75	39
2	1 488	74	39
3	8 008	75	39
4	1 488	74	39

Рисунок 1.5 – Повторное применение left (outer) join

```

-- подсчёт всех сумм, привязанных к транзакция
select
  "transaction".id_transaction,
  count(*) trans_sum
from
  transaction
inner join
  request
on
  transaction.id_app_user = request.id_app_user
where
  transaction.sum > 1000 and
  transaction.sum < 9000
group by
  "transaction".id_transaction

```

	123 id_transaction	123 trans_sum
1	74	2
2	75	2

Рисунок 1.6 – Повторное применение (outer) join

dbstud Script dbstud *dbstud> Sc... dbstud dbstud dbstud> Sc... dbstud

```
-- ко всем столбцам выбранным столбцам из transaction подсоединяются соответствующие
-- значения из request и наоборот при помощи full join
select
    transact.id_app_user ,
    req.id_transaction
from
    transaction as transact
full join
    request as req
on
    transact.id_app_user = req.id_app_user
```

transaction(+) 1 X

select transact.status, req.id_transaction f Введите SQL выражение чтобы отфильтровать результаты

	123 id_app_user	123 id_transaction
1	1	1
2	1	1
3	2	2
4	2	2
5	3	[NULL]
6	3	[NULL]
7	4	[NULL]
8	5	[NULL]
9	5	[NULL]
10	5	[NULL]
11	38	[NULL]
12	39	74
13	39	74
14	39	75
15	39	75

Рисунок 1.7 – Применение full join

dbstud Script dbstud *dbstud> Sc... dbstud dbstud dbstud> Sc... dbstud dbstud> Sc... dbstud request transaction

```
-- декартово произведение выбранных столбцов из таблиц transaction,
-- request и app_data при помощи cross join
select
    transact.id_app_user ,
    req.id_request ,
    app.personal_data
from
    request as req
cross join
    transaction as transact,
    app_user as app
```

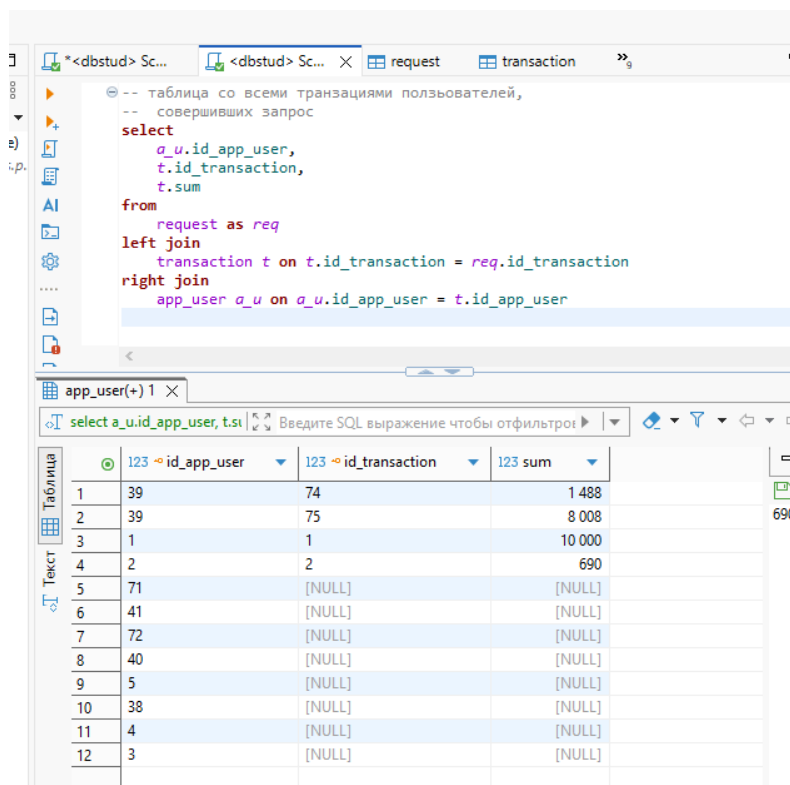
transaction(+) 1 X

select transact.id_app_user, req.id_request Введите SQL выражение чтобы отфильтровать результаты

	123 id_app_user	123 id_request	A2 personal_data
1	1	1	some data about user 1
2	2	1	some data about user 1
3	3	1	some data about user 1
4	4	1	some data about user 1
5	5	1	some data about user 1
6	1	1	some data about user 1
7	2	1	some data about user 1
8	3	1	some data about user 1
9	5	1	some data about user 1
10	5	1	some data about user 1
11	38	1	some data about user 1
12	39	1	some data about user 1
13	39	1	some data about user 1
14	1	2	some data about user 1
15	2	2	some data about user 1
16	3	2	some data about user 1
17	4	2	some data about user 1
18	5	2	some data about user 1
19	1	2	some data about user 1
20	2	2	some data about user 1
21	3	2	some data about user 1
22	5	2	some data about user 1
23	5	2	some data about user 1
24	38	2	some data about user 1
25	39	2	some data about user 1
26	39	2	some data about user 1
27	1	3	some data about user 1
28	2	3	some data about user 1
29	3	3	some data about user 1

Рисунок 1.8 – Применение cross join

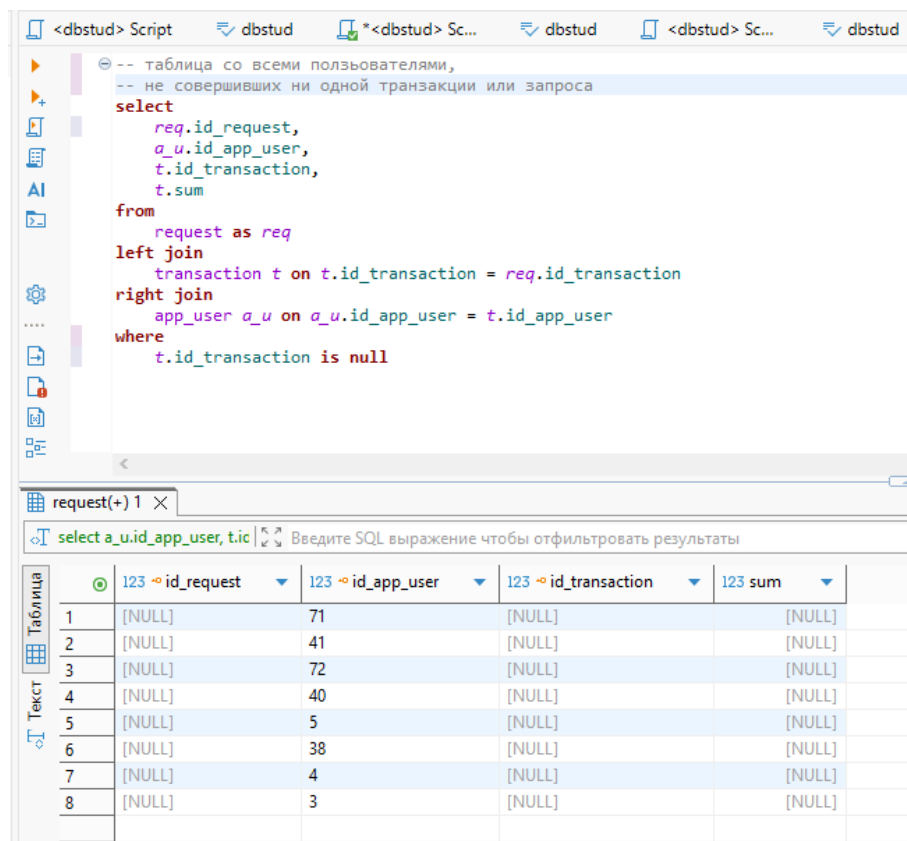
2. Продвинутые техники и паттерны соединений



```
-- таблица со всеми транзакциями пользователей,
-- совершивших запрос
select
    a_u.id_app_user,
    t.id_transaction,
    t.sum
from
    request as req
left join
    transaction t on t.id_transaction = req.id_transaction
right join
    app_user a_u on a_u.id_app_user = t.id_app_user
```

	id_app_user	id_transaction	sum
1	39	74	1 488
2	39	75	8 008
3	1	1	10 000
4	2	2	690
5	71	[NULL]	[NULL]
6	41	[NULL]	[NULL]
7	72	[NULL]	[NULL]
8	40	[NULL]	[NULL]
9	5	[NULL]	[NULL]
10	38	[NULL]	[NULL]
11	4	[NULL]	[NULL]
12	3	[NULL]	[NULL]

Рисунок 2.1 – Соединение трёх таблиц



```
-- таблица со всеми пользователями,
-- не совершивших ни одной транзакции или запроса
select
    req.id_request,
    a_u.id_app_user,
    t.id_transaction,
    t.sum
from
    request as req
left join
    transaction t on t.id_transaction = req.id_transaction
right join
    app_user a_u on a_u.id_app_user = t.id_app_user
where
    t.id_transaction is null
```

	id_request	id_app_user	id_transaction	sum
1	[NULL]	71	[NULL]	[NULL]
2	[NULL]	41	[NULL]	[NULL]
3	[NULL]	72	[NULL]	[NULL]
4	[NULL]	40	[NULL]	[NULL]
5	[NULL]	5	[NULL]	[NULL]
6	[NULL]	38	[NULL]	[NULL]
7	[NULL]	4	[NULL]	[NULL]
8	[NULL]	3	[NULL]	[NULL]

Рисунок 2.2 – Применение паттерна «Анти-соединение» (*anti-Join*)

transaction 1

select t.s.id_transaction, t.f.id_transaction from transaction t_f join transaction t_s on t_f.sum = t_s.sum and t_f.id_transaction != t_s.id_transaction order by t_s.id_transaction asc

	123 id_transaction	123 id_transaction
1	1	38
2	2	39
3	3	40
4	4	6
5	4	41
6	6	4
7	6	41
8	38	1
9	39	2
10	40	3
11	41	4
12	41	6

Рисунок 2.3 – Пример само-соединения

3. Теоретико-множественные операторы

Результат 1

select t.status from transaction t union select r.id_transaction from request r

	123 status
1	2
2	75
3	74
4	0
5	1

Рисунок 3.1 – Применение union

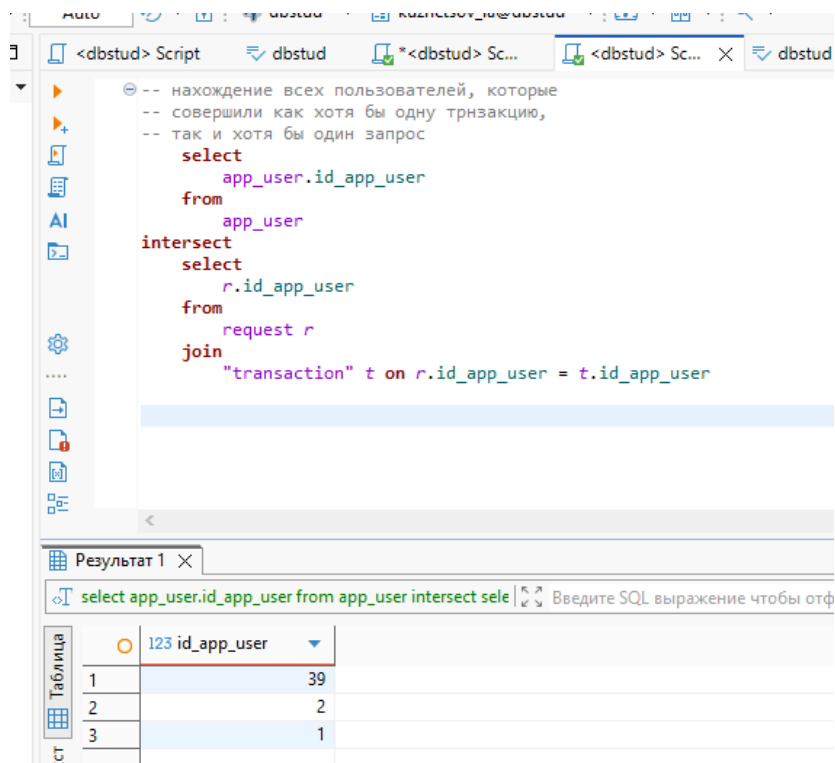


Рисунок 3.2 – применение intersect

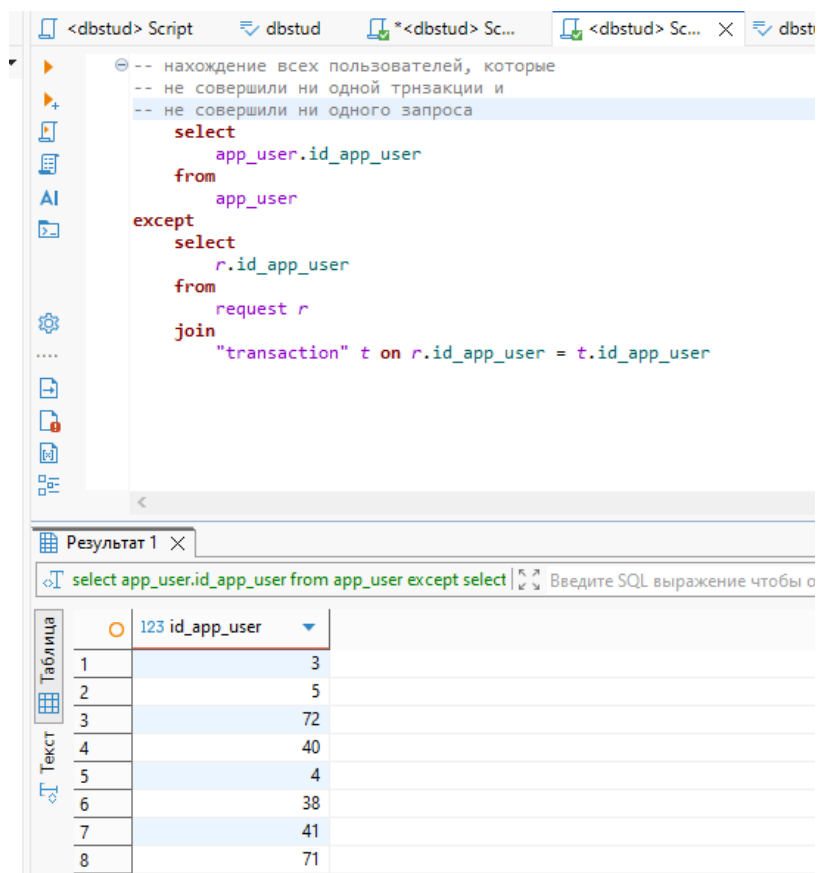
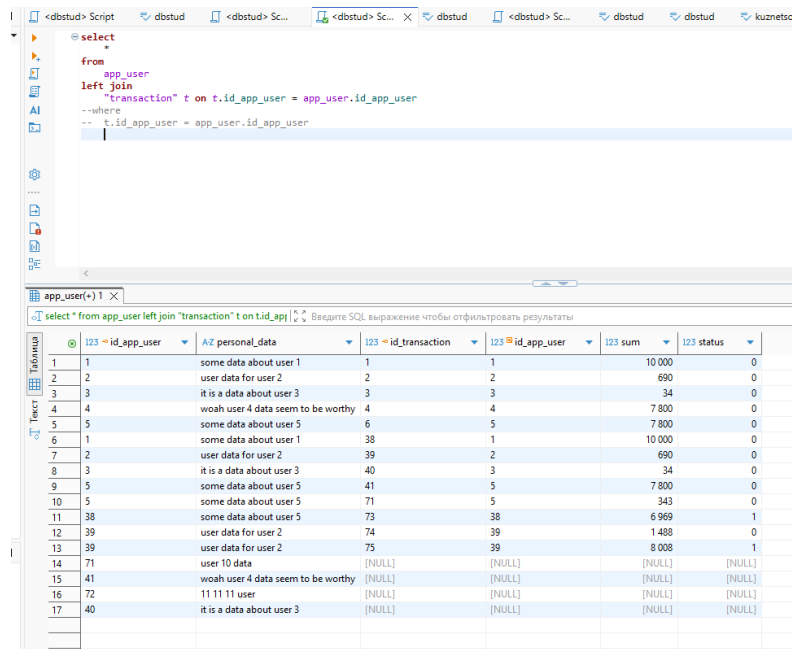


Рисунок 3.3 – Применение эксерт

ОТВЕТЫ НА ВОПРОСЫ

1. Нужно найти всех пользователей и запросы, связанные с ними;
2. оп производится во время join, а when – после. В качестве примера смотреть рисунки 4-6.
3. Потому что СУБД не будет знать, к какой из таблиц обращаться, что вызовет синтаксическую ошибку;
4. Когда мы работаем с заведомо уникальными данными;
5. Можно, для этого достаточно грамотно использовать ON. Пример с лекарствами;



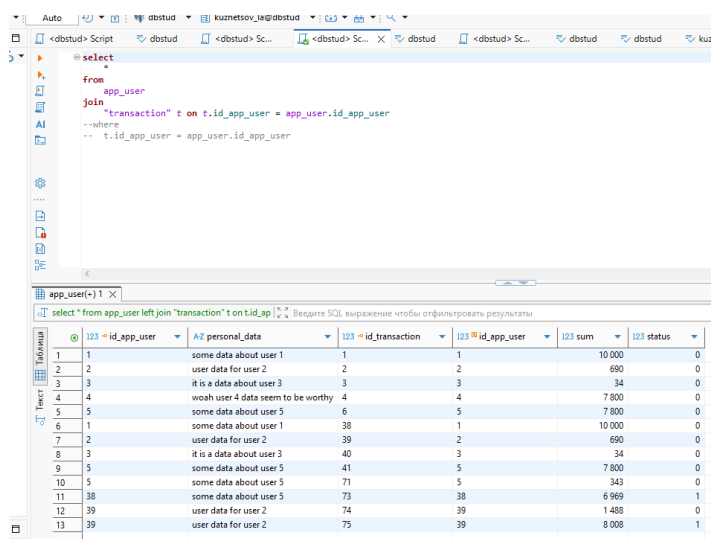
The screenshot shows a database query editor with a SQL query using a left join. The query is:

```
select
+
+
from
+
app_user
left join
+
"transaction" t on t.id_app_user = app_user.id_app_user
--where
+
-- t.id_app_user = app_user.id_app_user
```

The results table shows the following data:

	123 id_app_user	A2 personal_data	123 id_transaction	123 id_app_user	123 sum	123 status
1	1	some data about user 1	1	1	10 000	0
2	2	user data for user 2	2	2	690	0
3	3	it is a data about user 3	3	3	34	0
4	4	woah user 4 data seem to be worthy	4	4	7 800	0
5	5	some data about user 5	6	5	7 800	0
6	1	some data about user 1	38	1	10 000	0
7	2	user data for user 2	39	2	690	0
8	3	it is a data about user 3	40	3	34	0
9	5	some data about user 5	41	5	7 800	0
10	5	some data about user 5	71	5	343	0
11	38	some data about user 5	73	38	6 969	1
12	39	user data for user 2	74	39	1 488	0
13	39	user data for user 2	75	39	8 008	1
14	71	user 10 data	[NULL]	[NULL]	[NULL]	[NULL]
15	41	woah user 4 data seem to be worthy	[NULL]	[NULL]	[NULL]	[NULL]
16	72	11 11 11 user	[NULL]	[NULL]	[NULL]	[NULL]
17	40	it is a data about user 3	[NULL]	[NULL]	[NULL]	[NULL]

Рисунок 4 – Результат отбора при помощи left join



The screenshot shows a database query editor with a SQL query using an inner join. The query is:

```
select
+
+
from
+
app_user
join
+
"transaction" t on t.id_app_user = app_user.id_app_user
--where
+
-- t.id_app_user = app_user.id_app_user
```

The results table shows the following data:

	123 id_app_user	A2 personal_data	123 id_transaction	123 id_app_user	123 sum	123 status
1	1	some data about user 1	1	1	10 000	0
2	2	user data for user 2	2	2	690	0
3	3	it is a data about user 3	3	3	34	0
4	4	woah user 4 data seem to be worthy	4	4	7 800	0
5	5	some data about user 5	6	5	7 800	0
6	1	some data about user 1	38	1	10 000	0
7	2	user data for user 2	39	2	690	0
8	3	it is a data about user 3	40	3	34	0
9	5	some data about user 5	41	5	7 800	0
10	5	some data about user 5	71	5	343	0
11	38	some data about user 5	73	38	6 969	1
12	39	user data for user 2	74	39	1 488	0
13	39	user data for user 2	75	39	8 008	1

Рисунок 5 – Результат отбора при помощи inner join

```

select
  *
from
  app_user
left join
  "transaction" t on true
where
  t.id_app_user = app_user.id_app_user

```

	123 id_app_user	124 personal_data	123 id_transaction	123 id_app_user	123 sum	123 status
1	1	some data about user 1	1	1	10 000	0
2	2	user data for user 2	2	2	690	0
3	3	it is a data about user 3	3	3	34	0
4	4	woah user 4 data seem to be worthy	4	4	7 800	0
5	5	some data about user 5	6	5	7 800	0
6	1	some data about user 1	38	1	10 000	0
7	2	user data for user 2	39	2	690	0
8	3	it is a data about user 3	40	3	34	0
9	5	some data about user 5	41	5	7 800	0
10	5	some data about user 5	71	5	343	0
11	38	some data about user 5	73	38	6 969	1
12	39	user data for user 2	74	39	1 488	0
13	39	user data for user 2	75	39	8 008	1

Рисунок 6 – Результат отбора при помощи left join и where, вместо on

ВЫВОД

В ходе выполнения работы были получены базовые навыки для работы с join и теоретико-множественными операторами. Полученные знания были закреплены путём выполнения практического задания.